

CSC3066 Deep Learning

Challenges in training Deep Neural Networks

Anna Jurek-Loughrey

What's to come today

- ❑ Training Deep NNs: challenges and solutions
 - ❑ Vanishing/exploding gradient
 - ❑ Weight initialization
 - ❑ Long training time
 - ❑ Overfitting

Neural Networks:

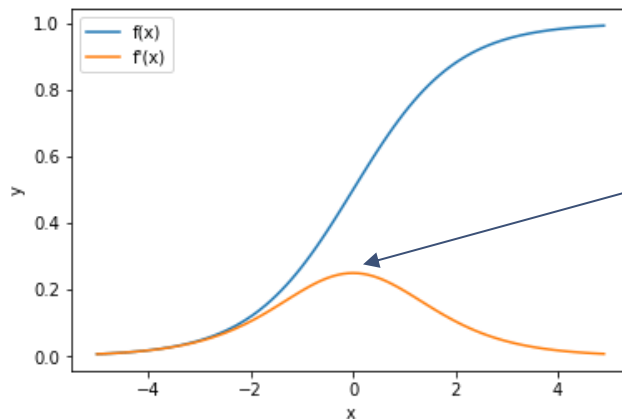
Vanishing/exploding gradients problem

Training NN: Vanishing/Exploding Gradients

- ❑ Vanishing or exploding gradient problem
 - ❑ The Gradient Descent algorithm uses the gradient of the cost function with regards to each parameter in the network to update their values during the training process.
 - ❑ During the backpropagation the gradients may get smaller and smaller as the algorithm progresses down to the lower layers. As a result, the weights of lower connections stay unchanged and training never coversages to a good solution. (Vanishing gradients problem)
 - ❑ In some cases, the gradients can grow bigger and bigger, so many layers get insanely large weight updates and the algorithm diverges. (Exploding gradients problem)

Training NN: Vanishing/Exploding Gradients

- ❑ Sigmoid: Vanishing Gradients Problem
 - ❑ The vanishing gradient problem is particularly problematic with Sigmoid activation function



The maximum point of the derivative function is $\frac{1}{4}$

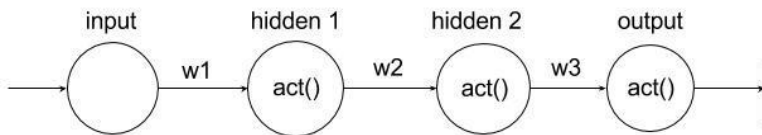
Training NN: Vanishing/Exploding Gradients

- ❑ Sigmoid: Vanishing Gradients Problem
 - ❑ What implication does the shape of the derivative function have on the backpropagation process?

Training NN: Vanishing/Exploding Gradients

- ❑ Sigmoid: Vanishing Gradients Problem
 - ❑ What implication does the shape of the derivative function have on the backpropagation process?
 - ❑ To calculate the derivative of w_1 the chain rule is applied:

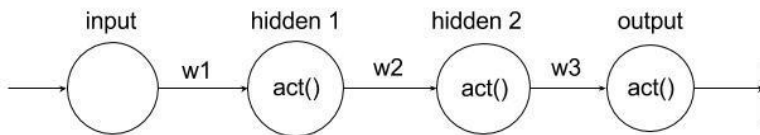
$$\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial out} \times \frac{\partial out}{\partial in} \times \frac{\partial in}{\partial out2} \times \frac{\partial out2}{\partial in2} \times \frac{\partial in2}{\partial out1} \times \frac{\partial out1}{\partial in1} \times \frac{\partial in1}{\partial w_1}$$



Training NN: Vanishing/Exploding Gradients

- ❑ Sigmoid: Vanishing Gradients Problem
 - ❑ What implication does the shape of the derivative function have on the backpropagation process?
 - ❑ To calculate the derivative of w_1 the chain rule is applied:

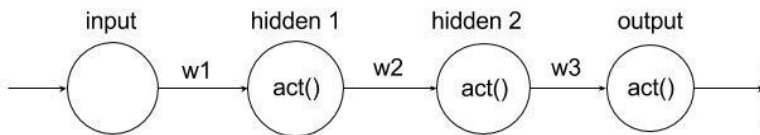
$$\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial out} \times \frac{\partial act}{\partial in} \times \frac{\partial in}{\partial out2} \times \frac{\partial act}{\partial in2} \times \frac{\partial in2}{\partial out1} \times \frac{\partial act}{\partial in1} \times \frac{\partial in1}{\partial w_1}$$



Training NN: Vanishing/Exploding Gradients

- ❑ Sigmoid: Vanishing Gradients Problem
 - ❑ What implication does the shape of the derivative function have on the backpropagation process?
 - ❑ To calculate the derivative of w_1 the chain rule is applied:

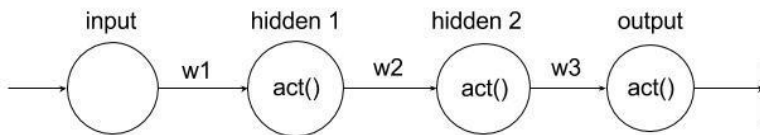
$$\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial out} \times \frac{\partial Sigm}{\partial in} \times \frac{\partial in}{\partial out2} \times \frac{\partial Sigm}{\partial in2} \times \frac{\partial in2}{\partial out1} \times \frac{\partial Sigm}{\partial in1} \times \frac{\partial in1}{\partial w_1}$$



Training NN: Vanishing/Exploding Gradients

- ❑ Sigmoid: Vanishing Gradients Problem
 - ❑ What implication does the shape of the derivative function have on the backpropagation process?
 - ❑ To calculate the derivative of w_1 the chain rule is applied:

$$\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial out} \times \frac{\partial Sigm}{\partial in} \times w_3 \times \frac{\partial Sigm}{\partial in2} \times w_2 \times \frac{\partial Sigm}{\partial in1} \times x$$



Training NN: Vanishing/Exploding Gradients

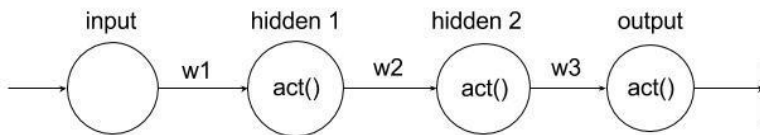
- ❑ Sigmoid: Vanishing Gradients Problem
 - ❑ What implication does the shape of the derivative function have on the backpropagation process?
 - ❑ To calculate the derivative of w_1 the chain rule is applied:

$$\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial out} \times \frac{\partial Sigm}{\partial in} \times w_3 \times \frac{\partial Sigm}{\partial in2} \times w_2 \times \frac{\partial Sigm}{\partial in1} \times x$$

$< 1/4$

$< 1/4$

$< 1/4$



Training NN: Vanishing/Exploding Gradients

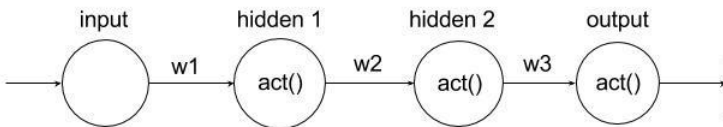
□ Sigmoid: Vanishing Gradients Problem

□ What implication does the shape of the derivative function have on the backpropagation process?

□ With standard approach to weight initialization, the weights are between -1 and 1.

$$\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial out} \times \frac{\partial Sigm}{\partial in} \times w_3 \times \frac{\partial Sigm}{\partial in2} \times w_2 \times \frac{\partial Sigm}{\partial in1} \times x$$

$$< 1/4 \quad < 1 \quad < 1/4 \quad < 1 \quad < 1/4$$



Training NN: Vanishing/Exploding Gradients

- ☐ Sigmoid: Vanishing Gradients Problem
 - ☐ What implication does the shape of the derivative function have on the backpropagation process?
 - ☐ During backpropagation very small values are multiplied.
 - ☐ The gradient becomes small very fast.
 - ☐ Even if we set the weights more carefully, in deeper NNs the vanishing gradient problem will still occur.

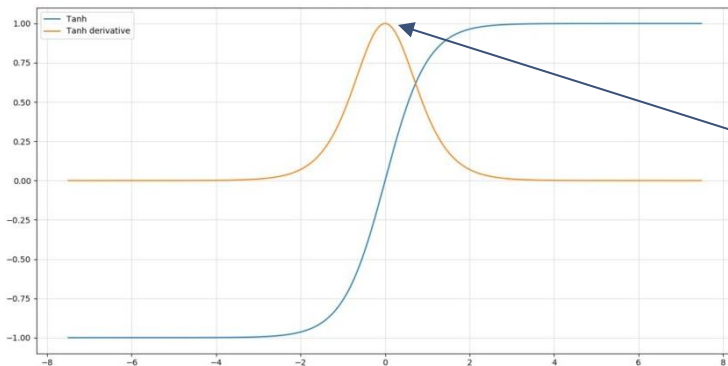
Training NN: Vanishing/Exploding Gradients

- ❑ Sigmoid: Vanishing Gradients Problem
 - ❑ The first layer is the furthest back from the error, so the derivative will be a longer expression and hence will contain more sigmoid derivatives.
 - ❑ Consequently, the first layer is the slowest to train.
 - ❑ The latter layers are dependent on the early layers. Inaccurate early layers may “corrupt” the entire network.

Training NN: Vanishing/Exploding Gradients

□ Tanh: Vanishing Gradients Problem

- The vanishing gradient problem also appears with the Tanh activation function



The maximum point of the derivative function is 1

Training NN: Vanishing/Exploding Gradients

- ❑ Both Sigmoid and Tanh activation functions can cause the vanishing gradient problem.
- ❑ This is the reason why Deep Neural Networks were not too popular at first.
- ❑ It was too difficult to train the early layers correctly because of the characteristics of the commonly used activation functions.

Neural Network:

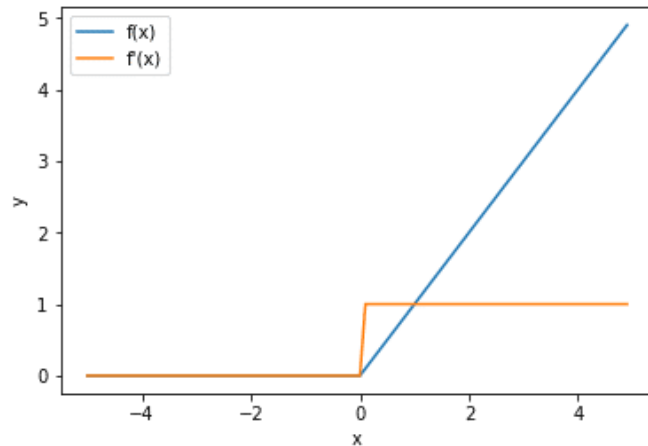
Addressing the vanishing gradient problem
with ReLu/Leaky ReLu activation

Training NN: Vanishing/Exploding Gradients

□ ReLu activation function:

$$\text{ReLu}(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x > 0 \end{cases}$$

$$\frac{\partial \text{ReLu}(x)}{\partial x} = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x > 0 \end{cases}$$



Training NN: Vanishing/Exploding Gradients

- ReLu activation function:

$$\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial out} \times \underbrace{\frac{\partial act}{\partial in}}_{\{0,1\}} \times \frac{\partial in}{\partial out2} \times \underbrace{\frac{\partial act}{\partial in2}}_{\{0,1\}} \times \frac{\partial in2}{\partial out1} \times \underbrace{\frac{\partial act}{\partial in1}}_{\{0,1\}} \times \frac{\partial in1}{\partial w_1}$$

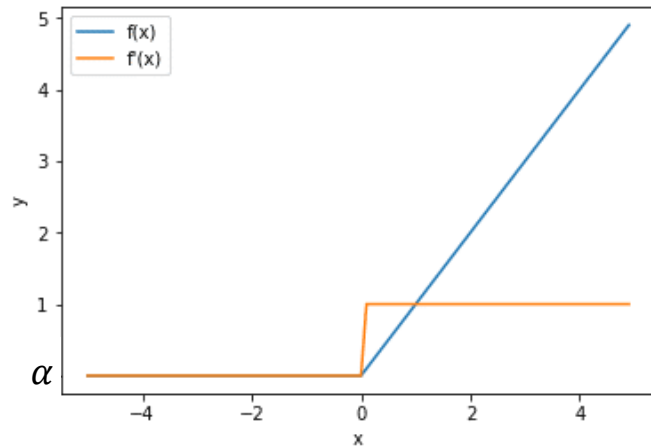
- Using ReLu activation function instead of Sigmoid/Tanh activation function in the hidden layers makes the gradient less likely to vanish.

Training NN: Vanishing/Exploding Gradients

□ ReLu activation function:

$$\text{Leaky ReLu}(x) = \begin{cases} \alpha x & \text{if } x < 0 \\ x & \text{if } x > 0 \end{cases}$$

$$\frac{\partial \text{Leaky ReLu}(x)}{\partial x} = \begin{cases} \alpha & \text{if } x < 0 \\ 1 & \text{if } x > 0 \end{cases}$$



Python Programming Example



Sigmoid vs Rely.ipynb

Neural Network:

Exploding Gradient Problem

Training NN: Vanishing/Exploding Gradients

- Let's consider a very deep Neural Network:

$$\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial out} \times \frac{\partial act}{\partial in} \times \frac{\partial in}{\partial outN} \times \dots \times \frac{\partial act}{\partial in2} \times \frac{\partial in2}{\partial out1} \times \frac{\partial act}{\partial in1} \times \frac{\partial in1}{\partial w_1}$$

- What will happen if the weights become larger than 1?
- What will happen if the error is very large?
- The gradient will become large!
- This will result in large updates to the network weights, and in turn, an unstable network.

Training NN: Vanishing/Exploding Gradients

- ❑ Signs of exploding gradient problem:
 - ❑ The model is unable to learn from the data (poor loss)
 - ❑ The model is unstable, i.e. large changes in loss from update to update
 - ❑ The model weights go to NaN

Training NN: Vanishing/Exploding Gradients

- ❑ Addressing the exploding gradient problem:
- ❑ **Gradient Clipping** - A popular technique to lessen the exploding gradient problem is to simply clip the gradient during backpropagation so that they never exceed some threshold (mainly used for Recurrent NN, later in the course).
- ❑ The algorithm first compute the gradients, clip them between $-threshold$ and $threshold$ and then apply them.
- ❑ The threshold is a hyperparameter of the algorithm and needs to be tuned.

Training NN: Vanishing/Exploding Gradients

- Addressing the exploding gradient problem (**Gradient Clipping**):
 - The threshold can be set via `clipvalue` argument and it can be used with all optimizers

```
from keras import optimizers

sdg = optimizers.SGD(lr = 0.01, clipvalue = 0.5)
model.compile(optimizer=sdg, loss='binary_crossentropy', metrics = ['acc'])
```

Training NN: Vanishing/Exploding Gradients

- ❑ Addressing the exploding gradient problem:
- ❑ **Weight Regularization:** Applying a penalty to the network loss function for large weight values → discuss in more details later in the slides.

Neural Network:

Weights Initialization

Weights Initialization

- ❑ Standard approaches to initializing the weights of Deep Neural Networks:
 1. Sampling from Normal Distribution (a probability distribution that is symmetric about the mean, showing that data near the mean are more frequent in occurrence than the data far from the mean.)
 2. Sampling from Uniform Distribution (a probability distribution in which all outcomes are equally likely i.e., each variable has the same probability that it will be the outcome.)

Weights Initialization

- Standard approaches to initializing the weights of Deep Neural Networks:

1. Sampling from Normal Distribution

```
from keras import initializers
init = initializers.RandomUniform(minval=-1, maxval=1)

model = Sequential()
model.add(Dense(5, input_dim=2, activation='relu', kernel_initializer=init))
...
```

2. Sampling from Uniform Distribution

```
from keras import initializers
init = initializers.RandomNormal(mean=0.0, stddev=0.1, seed=None)

model = Sequential()
model.add(Dense(5, input_dim=2, activation='relu', kernel_initializer=init))
...
```

Weights Initialization

- ❑ Problem with random weight initialisation:
 1. The variance of the output of each layer is much greater than the variance of its input.
 2. Going forward in the network, the variance keeps increasing after each layer, which may cause the activation function to explode at the top layers.
 3. This is a problem in particular with activation function which outputs are not bounded such as ReLu/leaky ReLu

Weights Initialization



Exploding activation issue.ipynb

Weights Initialization

□ Solution:

Activation Function	Uniform Distribution $[-r, r]$	Normal Distribution $(0, \sigma)$	
Sigmoid	$r = \sqrt{\frac{6}{n_{in} + n_{out}}}$ (glorot_uniform)	$\sigma = \sqrt{\frac{2}{n_{in} + n_{out}}}$ (glorot_normal)	Xavier Initialisation
ReLU/Leaky ReLU	$r = \sqrt{2} \times \sqrt{\frac{6}{n_{in} + n_{out}}}$ (he_uniform)	$\sigma = \sqrt{2} \times \sqrt{\frac{2}{n_{in} + n_{out}}}$ (he_normal)	He Initialisation

Where n_{in}/n_{out} are the number of input and output connections for the layer whose weights are being initialised

Weights Initialization

- ❑ Default initialization:
 - ❑ `kernel_initializer='glorot_uniform'`
 - ❑ `bias_initializer='zeros'`

```
from keras import initializers

init_x_n = keras.initializers.glorot_normal(seed=None)
init_he_u = keras.initializers.he_uniform(seed=None)
inin_he_n = keras.initializers.he_normal(seed=None)

model = Sequential()
model.add(Dense(5, input_dim=2, activation='relu', kernel_initializer=init_he_u))
```

Neural Network:

Speeding up Training Time:

Faster Optimisers

Speeding up Training Time: Faster Optimisers

- ❑ Problem with Gradient Descent:

$$W \leftarrow W - \nabla_W E(W)$$

1. Takes time to compute the gradients at every epoch, which makes it very slow when the training set is large.

Speeding up Training Time: Faster Optimisers

- ❑ Solution 1: Stochastic Gradient Descent:
 - ❑ Instead of using the entire training set, it picks a random instance at each step and computes the gradients based only on that single instance.
 - ❑ This makes the algorithm learn much faster.
 - ❑ Due to its stochastic (random) nature, it is much less regular. Instead of gently decreasing until it reaches minimum, the cost function will bounce up and down, decreasing only on average.

Speeding up Training Time: Faster Optimisers

- ❑ Solution 2: Mini-Batch Gradient Descent:
 - ❑ At each step instead of computing the gradients based on the full training set or based on just one instance, it computes the gradients on small random set of instances called mini-batches.
 - ❑ It can get closer to the minimum than Stochastic GD.
 - ❑ It is harder for it to escape local minima.

Speeding up Training Time: Faster Optimisers

❑ Problem with Gradient Descent:

$$W \leftarrow W - \alpha \nabla_W E(W)$$

2. In each iteration, it updates the weights W by directly subtracting the gradient of the cost function $E(W)$ with regards to the weights.
 - It does not care about the values of previous gradients.
 - If the local gradient is tiny, it goes very slow.

❑ There are some faster optimisers than Gradient Descent!

Speeding up Training Time: Faster Optimisers

- There are other optimisation techniques, which improves upon GD by applying small tweaks:
 - Nesterov Accelerated Gradient
 - AdaGrad
 - RMSProp
 - Adam
- We will skip the mathematical details of these.

<https://towardsdatascience.com/10-gradient-descent-optimisation-algorithms-86989510b5e9>

Weights Initialization



SGD vs Adam.ipynb

Neural Network:

Avoiding Overfitting

Avoiding Overfitting

- ❑ Deep NNs typically have tens of thousands of parameters.
- ❑ With so many parameters, the network has an incredible amount of freedom and can fit a huge variety of complex datasets.
- ❑ This great flexibility also means that it is prone to overfitting the training set.

Avoiding Overfitting

- ❑ Techniques for preventing overfitting:
 - ❑ Simplifying the model
 - ❑ Early Stopping
 - ❑ Regularization
 - ❑ Dropout

Avoiding Overfitting: Early Stopping

- ☐ **Early Stopping:** Interrupt training of a model when its performance on the validation set starts dropping.
 - ☐ E.g. Evaluate the model on validation set at regular intervals (e.g. 50 steps).
 - ☐ Save the model if it outperforms previous best model.
 - ☐ Interrupt the training when the number of steps since the last best model reaches some limit (e.g. 2000 steps).

Avoiding Overfitting: Early Stopping

□ Early Stopping:

```
from keras.callbacks import EarlyStopping

model = Sequential()
model.add(Dense(5, input_dim=2, activation='relu', kernel_initializer='he_normal'))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='sgd', metrics=['acc'])

#configure early stopping
es = EarlyStopping(monitor='val_loss', patience=5)

history = model.fit(trainX, trainy, validation_data=(testX, testy), epochs=500, verbose=0, callbacks = [es])
```

- monitor: quantity to be monitored.
- patience: number of epochs that produced the monitored quantity with no improvement after which training will be stopped.

Avoiding Overfitting: Regularization

- ❑ **Regularization:** adding penalty to the loss function to constrain a NN's connection weights.
- ❑ The most common techniques are known as l_1 or l_2 regularization:
- ❑ l_1 regularization: The penalty aims to minimize the absolute value of weights

$$l_1 = \sum_{i=1}^n |w_i|$$

Avoiding Overfitting: Regularization

- ❑ **Regularization:** adding penalty to the loss function to constrain a NN's connection weights.
- ❑ The most common techniques are known as l_1 or l_2 regularization:
- ❑ l_2 regularization: The penalty aims to minimize the squared magnitude of the weights.

$$l_2 = \sum_{i=1}^n (w_i)^2$$

Avoiding Overfitting: Regularization

- The regularization term can be added to the error/loss function:

$$L(x, y) = Loss(W) + \alpha \sum_i \sum_j |w_{ij}|$$

$$L(x, y) = Loss(W) + \alpha \sum_i \sum_j (w_{ij})^2$$

- α is the regularization parameter.

Avoiding Overfitting: Regularization

Regularization in Keras:

```
from keras.regularizers import l1

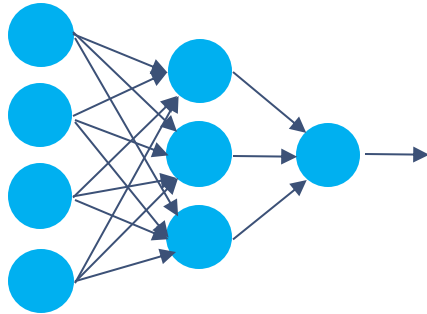
model = Sequential()
model.add(Dense(5, input_dim=2, activation='relu', kernel_regularizer=l1(0.001)))
```

Avoiding Overfitting: Dropout

- ❑ **Dropout:** At every training step, every neuron has a probability p of being temporally “dropped out” (it will be ignored in the training step but it may be activated during the next step).
- ❑ The parameter p is called the dropout rate, and it is typically set to 50%.
- ❑ After training, the neurons don’t get dropped out anymore.

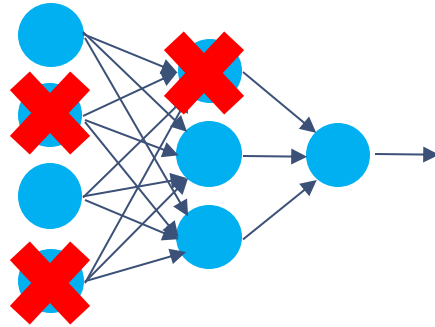
Avoiding Overfitting: Dropout

Example of single dropout step:



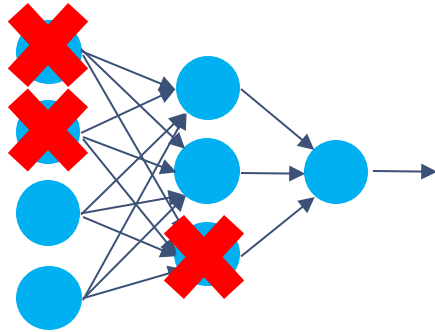
Avoiding Overfitting: Dropout

Example of single dropout step:



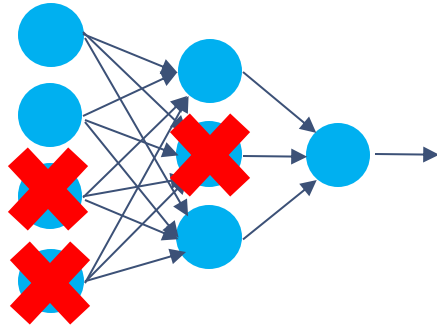
Avoiding Overfitting: Dropout

Example of single dropout step:



Avoiding Overfitting: Dropout

Example of single dropout step:



Avoiding Overfitting: Dropout

Dropout in Kares:

In Keras, we can implement dropout by adding Dropout layers into the network architecture. Each Dropout layer will drop a user-defined hyperparameter of units in the previous layer every batch.

```
from keras.layers import Dropout

model = Sequential()
model.add(Dense(15, input_dim=20, activation='relu', kernel_regularizer=l1(0.001)))
model.add(Dropout(0.2))
model.add(Dense(10, activation='relu', kernel_regularizer=l1(0.001)))
model.add(Dropout(0.2))
model.add(Dense(10, activation='relu', kernel_regularizer=l1(0.001)))
```


Avoiding Overfitting: Dropout

Dropout in Kares:

The input layer is assumed to be the first layer. Therefore, if we want to add dropout to the input layer, the layer we add in first is a dropout layer. This layer contains both the proportion of the input layer's units to drop and `input_shape` defining the shape of the observation data.

```
from keras.layers import Dropout

model = Sequential()
model.add(Dropout(0.2, input_shape=(20,)))
model.add(Dense(15, activation='relu', kernel_regularizer=l1(0.001)))
model.add(Dropout(0.2))
model.add(Dense(10, activation='relu', kernel_regularizer=l1(0.001)))
model.add(Dropout(0.2))
model.add(Dense(10, activation='relu', kernel_regularizer=l1(0.001)))
```

Coming Next:

Working with text data